2026-9-17 Linux网络--UDP理论

- 1. UDP 协议理论
  - 1.1 UDP 在协议栈中的位置
    - 1.1.1 从“主机通信”到“进程通信”
    - 1.1.2 UDP 的基本性质
  - 1.2 UDP 数据报格式
    - 1.2.1 固定 8 字节报头
    - 1.2.2 报头分离与有效载荷分用
    - 1.2.3 为什么 UDP 不存在粘包问题
  - 1.3 封装、解包与内核报文
    - 1.3.1 封装的本质
    - 1.3.2 什么是内核中的报文
    - 1.3.3 报文为什么能通过文件接口被应用读取
- 2. UDP Socket 编程
  - 2.1 UDP 服务的通用模型
    - 2.1.1 创建 UDP socket
    - 2.1.2 服务端的基本流程
    - 2.1.3 客户端是否需要 bind
  - 2.2 地址与字节序
    - 2.2.1 sockaddr_in 与网络字节序
    - 2.2.2 INADDR_ANY 的含义
    - 2.2.3 字符串与二进制地址的转换
  - 2.3 从回显服务到实际应用
    - 2.3.1 Echo Server：最小闭环
    - 2.3.2 DictServer：请求、处理与响应
    - 2.3.3 ChatServer：共享一个 socket 处理多个客户端
  - 2.4 UDP 服务实现要点
    - 2.4.1 地址、长度和缓冲区
    - 2.4.2 “数据报边界”不等于“业务消息可靠”
    - 2.4.3 抽取可复用的 UDP 封装
  - 2.5 本章小结
- 3. 登录、Cookie 与 Session
  - 3.1 一次登录请求如何变成持续会话
    - 3.1.1 登录页与身份认证
    - 3.1.2 服务器端如何保存登录状态
    - 3.1.3 Set-Cookie 与后续请求

    - 3.2 Cookie 与 Session 的职责边界
        - 3.2.1 不建议把私密信息直接放进 Cookie
        - 3.2.2 注销、过期与失效
    - 3.3 Web 会话与操作系统会话的联系
- 4. 终端会话与进程管理
    - 4.1 进程组、作业与 Session
        - 4.1.1 进程组与作业
        - 4.1.2 作业号与 jobs
        - 4.1.3 进程组与进程组长
    - 4.2 前台与后台作业
        - 4.2.1 一个终端会话的输入权
        - 4.2.2 fg、bg、Ctrl+C 与 Ctrl+Z
    - 4.3 SID 与终端登录会话
        - 4.3.1 登录会创建 Session
        - 4.3.2 注销会话为什么会影响服务
- 5. 网络服务守护进程化
    - 5.1 为什么要创建独立 Session
    - 5.2 setsid 与守护进程
        - 5.2.1 setsid 做了什么
        - 5.2.2 守护进程的工程含义
    - 5.3 标准流与日志
        - 5.3.1 重定向标准输入输出
        - 5.3.2 通过日志观察服务
    - 5.4 部署目录与路径管理
        - 5.4.1 模块化目录
        - 5.4.2 使用稳定的起始路径
    - 5.5 从登录 Session 到独立网络服务

1. UDP 协议理论

UDP（User Datagram Protocol，用户数据报协议）位于传输层。它向下使用 IP 完成主机到主机的数据传输，向上通过端口号把数据交还给具体进程。与面向字节流的协议相比，UDP 每一次发送都会形成一个独立的数据报，接收方一次读取得到一个完整边界内的数据报。

> 本文先整理 UDP 协议与 Socket 编程，再补上登录认证、Cookie、Session、终端进程管理和网络服务守护进程化的内容，最后把“协议通信”与“服务如何长期运行”连接起来。

1.1 UDP 在协议栈中的位置

1.1.1 从“主机通信”到“进程通信”

IP 地址能够定位网络中的一台主机，但一台主机上可以同时运行多个网络进程。因此，仅凭 IP 地址还无法决定数据应该交给哪个程序，还需要传输层端口号。

UDP 使用 16 位源端口和 16 位目的端口，所以端口号范围由协议头字段决定。发送端通过源端口标识数据报来自哪个进程，接收端则根据目的端口完成向上交付。

可以把这个过程概括为：

1. IP 层负责把数据送到目标主机；
2. UDP 根据目的端口找到目标进程；
3. 应用层通过 socket 接口读取这个数据报。

1.1.2 UDP 的基本性质

UDP 不建立类似 TCP 三次握手的连接，也不在协议内部维护确认、重传、排序等可靠传输机制。因此，UDP 具有以下特点：

- **无连接**：每次读写都围绕数据报和对端地址进行，不需要先建立持久连接；
- **面向数据报**：一次发送对应一个独立数据报，接收端能够识别数据报边界；
- **不保证可靠交付**：网络拥塞、链路异常等情况可能造成丢包，UDP 本身不负责补发；
- **开销小、处理直接**：协议头固定且逻辑简单，适合能够容忍个别数据报丢失、但重视时延和实现简单性的场景。

“不可靠”并不是说 UDP 一定传不到，而是说**UDP 不向应用承诺可靠交付**。如果业务要求消息不丢失、不重复或严格有序，就需要在应用层补充对应机制。

1.2 UDP 数据报格式

1.2.1 固定 8 字节报头

UDP 报头长度固定为 8 字节，由四个 16 位字段组成：

比特范围	字段	作用
0-15	16 位源端口号	标识发送方进程；不需要回包时也可以为 0
16-31	16 位目的端口号	接收端据此完成向上分用
32-47	16 位 UDP 长度	表示 UDP 报头和有效载荷的总长度
48-63	16 位 UDP 校验和	用于检验数据在传输过程中是否发生错误

其结构在 C 语言中可以直观理解为：

```
struct udphdr {
    uint16_t source;
    uint16_t dest;
    uint16_t len;
    uint16_t check;
};
```

操作系统内部的协议实现也使用结构体描述这些字段。由于发送端和接收端遵循同一个协议标准，即使两台主机的操作系统不同，也可以正确解释同一份报头。

1.2.2 报头分离与有效载荷分用

接收端拿到一个 UDP 数据报后，需要回答两个不同的问题：

1. **报头与有效载荷如何分开？**
   UDP 报头长度固定为 8 字节，接收端先按固定位置读取报头，剩余部分就是有效载荷。`16 位 UDP 长度`还可以帮助确定这个数据报的实际总长度。
2. **有效载荷应该交给哪个进程？**
   读取 `16 位目的端口号`，再根据端口号找到对应的 socket，把有效载荷向上交付给该进程。

因此，UDP 使用“固定长度报头 + 长度字段”完成边界识别，使用“目的端口”完成向上分用。

1.2.3 为什么 UDP 不存在粘包问题

UDP 是用户数据报协议，而不是字节流协议。发送方每次调用发送接口，接收方收到的就是一个具有独立边界的数据报；即使应用连续发送两条消息，接收端也可以通过多次读取分别得到它们，不会像字节流那样天然合并成无法区分边界的一串数据。

需要区分的是：

- **消息边界**由 UDP 数据报机制保留；
- **消息内部字段如何组织**仍要由应用自己定义，例如用空格、分隔符或结构化编码表示多个字段；
- **数据报是否到达、先后顺序是否正确**仍不在 UDP 的保证范围内。

所以，“UDP 没有粘包问题”不等于“应用可以省略协议设计”。它只是省去了应用层根据长度或分隔符重新切分多条消息的步骤。

1.3 封装、解包与内核报文

1.3.1 封装的本质

应用层调用 socket 发送接口后，数据会沿协议栈逐层向下传递。对于 UDP 来说，传输层需要构造 UDP 报头，并把源端口、目的端口、长度和校验和填写到相应字段中。

从数据结构角度看，封装可以理解为对协议结构体字段的填写和变量复制。例如用下面的方式描述报头字段的赋值：

```
((struct udphdr *)q)->source = source_port;
((struct udphdr *)q)->dest   = dest_port;
((struct udphdr *)q)->len    = udp_length;
((struct udphdr *)q)->check  = checksum;
```

数据向下传递时，下层协议继续在已有数据前面加入自己的协议头；接收端向上传递时，则逐层识别并剥离本层报头。这里的核心不只是“增加一块内存”，而是通过指针移动界定每一层的头部和数据区。

1.3.2 什么是内核中的报文

操作系统内核可能同时维护很多正在发送或等待接收的报文，因此必须做到“先描述，再组织”。

Linux 中用于描述网络数据包的核心结构是 `sk_buff`。一个报文不能简单理解成一块独立缓冲区，它更接近下面两部分的组合：

- 描述报文状态、长度、所属设备、队列关系等信息的结构体；

- 实际存放各层协议头和应用数据的内核缓冲区。

`sk_buff` 通过多个指针管理这块缓冲区，例如：

- `head`：缓冲区起始位置；
- `data`：当前协议层有效数据的起始位置；
- `tail`：当前有效数据的结束位置；
- `end`：缓冲区结束位置。

当数据向下封装时，可以在头部空间写入本层协议头，并移动 `data`；当数据向上解包时，识别并移除本层协议头后，同样通过调整 `data` 指向下一层数据。这样，报文穿过协议栈时不必每次都复制全部有效载荷，核心操作可以落在指针移动和少量字段更新上。

多个报文之间可以通过队列组织起来，便于内核完成插入、删除和查找。这也是“先描述、再组织”在协议实现中的具体体现。

1.3.3 报文为什么能通过文件接口被应用读取

在 Linux 中，socket 也遵循“一切皆文件”的思想。应用拿到的是一个文件描述符，但这个整数背后有一组内核对象：

```
文件描述符 fd
    -> struct file
    -> private_data
    -> struct socket
    -> struct sock
    -> 对应协议的具体实现
```

应用调用 `recvfrom` 时，内核通过文件描述符找到对应的 `struct file`，再通过 `private_data` 找到网络 socket，最终执行 UDP 层的接收逻辑。发送时则按相反方向，从应用缓冲区逐层封装并交给网卡。

从课堂模型看，数据向上交付时会触发上层注册的回调，向下发送时则调用下层函数。应用之所以能够用文件读写风格操作网络，是因为 VFS 的文件抽象和 socket 私有数据把“文件操作”和“网络协议实现”连接了起来。

2. UDP Socket 编程

2.1 UDP 服务的通用模型

2.1.1 创建 UDP socket

IPv4 UDP socket 通过下面的调用创建：

```
int fd = socket(AF_INET, SOCK_DGRAM, 0);
```

其中：

- `AF_INET` 表示 IPv4；
- `SOCK_DGRAM` 表示数据报套接字；
- 最后一个参数通常写 0，由系统选择对应的 UDP 协议实现。

UDP 不需要 `listen` 和 `accept`。服务端创建 socket 后，重点是把本地地址和端口绑定到 socket 上，然后进入循环等待数据报。

2.1.2 服务端的基本流程

UDP 服务端最核心的流程是：

```
socket() -> bind() -> recvfrom() -> 处理请求 -> sendto() -> 继续接收
```

服务端必须显式绑定一个已知端口。客户端只有知道服务器的 IP 地址和端口，才能把请求发送到正确进程。

一个典型的接收和回复过程如下：

```
struct sockaddr_in peer;
socklen_t len = sizeof(peer);
char buffer[1024];

ssize_t n = recvfrom(fd, buffer, sizeof(buffer) - 1, 0,
                     (struct sockaddr *)&peer, &len);
if (n > 0) {
    buffer[n] = '\0';
    sendto(fd, buffer, strlen(buffer), 0,
```

```
            (struct sockaddr *)&peer, len);
}
```

这里需要注意几点：

- `peer` 由一次接收操作填入，表示这个数据报来自哪个地址和端口；
- 回复时使用刚刚得到的 `peer`，因此同一个 socket 可以依次响应不同客户端；
- 文本处理时通常预留一个字节写入 `'\0'`，所以接收缓冲区长度传 `sizeof(buffer) - 1`；
- 如果处理的是二进制数据，发送长度应使用实际接收长度 `n`，不能把 `'\0'` 或旧数据一起发送；
- `recvfrom` 和 `sendto` 的返回值都应检查，不能默认每次收发一定成功。

2.1.3 客户端是否需要 bind

客户端通常不需要显式调用 `bind`。第一次通过 UDP socket 发送数据时，内核会自动为它选择本机的临时源 IP 和源端口，这种行为也叫隐式绑定。

客户端一般不确定未来会从哪个网卡发出请求，或者本机可能有多个 IP 地址，因此把源地址交给操作系统选择更方便。显式绑定只在需要固定源端口、配合防火墙规则或实现特殊协议行为时才有必要。

服务端与客户端的差异可以概括为：

角色	是否通常 bind	原因
服务端	需要	服务地址和端口必须公开、稳定，客户端才能找到它
客户端	通常不需要	源地址和临时端口可由内核在首次发送时自动确定

2.2 地址与字节序

2.2.1 sockaddr_in 与网络字节序

IPv4 地址通常使用 `sockaddr_in` 描述：

```
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = INADDR_ANY;
```

其中：

- `sin_family` 设置为 `AF_INET`；
- `sin_port` 必须使用网络字节序，传递端口时通常使用 `htons`；
- `sin_addr` 保存 32 位 IPv4 地址；
- `htons`、`ntohs` 分别用于主机字节序和网络字节序之间的转换；
- `inet_addr` 可以把点分十进制 IP 字符串转换为网络字节序的 32 位地址。

绑定前要先初始化整个结构体，避免未初始化字段影响调用。

2.2.2 INADDR_ANY 的含义

`INADDR_ANY` 的值为 0，表示绑定本机所有可用网络接口：

```
addr.sin_addr.s_addr = INADDR_ANY;
```

服务器把地址写成 `INADDR_ANY` 后，无论请求从本机的哪张网卡进入，只要目的端口匹配，都可以交给这个 socket 处理。这样就不需要提前写死某一台虚拟机或云服务器的公网 IP。

特别强调：云服务器通常不允许直接绑定某个公网 IP，服务端编写时也不推荐写死明确 IP，使用 `INADDR_ANY` 更稳妥。

如果服务器绑定了具体的 IP 地址，那么它只接收发往该地址的数据；目的地址是其他本机 IP 的数据报不会由这个 socket 接收。

2.2.3 字符串与二进制地址的转换

常见地址转换函数包括：

- `inet_addr`：IPv4 点分十进制字符串转换为 `in_addr`；
- `inet_ntoa`：IPv4 `in_addr` 转换为点分十进制字符串；

- `inet_pton` / `inet_ntop`：支持 IPv4 和 IPv6 的通用转换接口。

`inet_ntoa` 虽然使用方便，但它把结果放在函数内部的静态缓冲区中。再次调用时，上一次返回的字符串可能被覆盖。因此：

```
char *p1 = inet_ntoa(addr1.sin_addr);
char *p2 = inet_ntoa(addr2.sin_addr);
```

`p1` 和 `p2` 最终可能指向同一块静态存储区，导致两条地址显示成同一个值。更重要的是，`inet_ntoa` 不是线程安全接口。

在多线程环境中，推荐使用 `inet_ntop`，由调用者提供输出缓冲区：

```
char ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &addr.sin_addr, ip, sizeof(ip));
```

这样每个线程使用自己的结果空间，不会因为共享静态缓冲区发生覆盖。

2.3 从回显服务到实际应用

2.3.1 Echo Server：最小闭环

实现一个简单的回显服务器：

1. 创建 UDP socket；
2. 服务端绑定固定端口；
3. 循环调用 `recvfrom` 接收数据报；
4. 把收到的内容通过 `sendto` 发回源地址；
5. 客户端再用 `recvfrom` 接收回复。

这个例子把 UDP 通信的最小闭环完整串了起来：服务端必须绑定，客户端可以先不绑定；服务端每次处理请求时都会从 `peer` 中得到客户端地址，并据此回复。

Echo Server 也说明了一个重要事实：UDP socket 是全双工的。同一个描述符既可以接收，也可以发送，不需要像单向管道那样创建两个端口。

2.3.2 DictServer：请求、处理与响应

V2 版本把 Echo Server 扩展成英译汉字典服务：

- 服务启动时从 `dict.txt` 读取词条和解释；
- `recvfrom` 收到的是待查询单词；
- 服务端在字典中查找，生成响应；
- 再通过 `sendto` 把结果发给刚才的客户端。

这种结构本质上是“接收请求 - 执行业务处理 - 发送响应”。为了提高复用性，讲义把业务处理抽成回调 `func_t`，UDP 服务端只负责网络收发，字典、聊天室等应用只需要提供各自的处理函数。

这也体现了网络层与业务层的分离：

```
UdpServer：负责 socket、地址、接收和发送
回调函数：负责解释请求并生成响应
```

UDP 保留了数据报边界，因此字典服务可以把“一个数据报”当作“一次查询”。不过，如果以后一个请求需要携带多个字段，仍然必须在应用数据内部定义字段格式。

2.3.3 ChatServer：共享一个 socket 处理多个客户端

V3 版本在字典服务的基础上实现了简单聊天室：

- 服务端维护在线用户的地址；
- 路由模块把客户端加入或移出在线列表；
- 收到消息后，由路由逻辑决定转发给其他哪些用户；
- 服务端可以用线程池并发处理多个请求。

聊天室进一步说明，UDP socket 并没有与某个客户端建立固定连接。服务端用同一个绑定端口的 socket 接收来自不同地址的数据报，再根据每个数据报携带的源地址分别回复或转发。

多线程客户端一边执行接收循环，一边执行发送循环，二者共用 UDP socket。因为 UDP 支持全双工，一个 socket 同时进行发送和接收是合理的，但共享数据结构时仍必须处理线程同步问题，不能认为“UDP 本身无连接”就等于“多线程访问天然安全”。

2.4 UDP 服务实现要点

2.4.1 地址、长度和缓冲区

- 服务端应绑定 `INADDR_ANY + 已知端口`，避免把公网 IP 写死；
- 客户端通常让内核自动选择源地址和源端口；
- `recvfrom` 中的 `socklen_t` 必须先初始化为 `sizeof(peer)`；
- 回复时要使用 `recvfrom` 填出的完整 `peer` 和对应长度；
- 接收文本时预留终止符空间，计算长度时不要越界；
- 二进制数据应按实际接收字节数发送，不要误用 `strlen`。

2.4.2 “数据报边界”不等于“业务消息可靠”

UDP 帮应用保留了单次发送的边界，但不会替应用解决丢失、重复和乱序。因此，在 UDP 之上实现重要业务时，需要考虑：

- 是否需要请求序号或消息 ID；
- 是否需要应用层确认与超时重传；
- 是否需要去重和重新排序；
- 一个数据报最多能承载多少应用数据；
- 对端断开时，UDP 没有连接关闭过程，业务如何判断对方已经离开。

Echo、字典查询这类请求-响应模型可以容忍偶尔丢失；聊天室、控制指令等场景则要根据业务重要性决定是否在应用层补足可靠性。

2.4.3 抽取可复用的 UDP 封装

从 V1 到 V3 逐步抽取了 `UdpSocket`、`UdpServer` 和 `UdpClient`：

- `UdpSocket` 封装创建、绑定、收发和关闭；
- `UdpServer` 封装“服务器循环 + 业务回调”；
- `UdpClient` 封装默认服务端地址和请求/响应流程；
- 应用层只保留字典或聊天室自己的业务规则。

这种分层的价值在于：网络接口、地址转换和错误检查集中维护，具体业务不需要重复书写 `socket`、`bind`、`recvfrom` 和 `sendto`。但封装不应隐藏关键语义，例如数据报边界、实际接收长度、对端地址和错误返回值。

2.5 本章小结

UDP 的核心可以从三个层次理解：

1. **协议层**：固定 8 字节报头保存端口、长度和校验和，目的端口完成向上分用，数据报边界天然保留；
2. **内核层**：`sk_buff` 描述内核报文并管理缓冲区，协议栈通过移动指针完成封装和解包，socket 通过文件抽象暴露给应用；
3. **编程层**：服务端 `socket -> bind -> recvfrom -> sendto`，客户端通常隐式绑定后直接发送；业务处理通过回调与网络收发分离。

理解 UDP 时既不能只把它看作“不安全的 TCP”，也不能因为实现简单就忽略地址、字节序、长度和线程安全。它提供的是一组边界清晰、但不保证可靠交付的数据报；是否需要可靠性，应由应用目标来决定。

3. 登录、Cookie 与 Session

HTTP 请求本身是无状态的，每个请求默认都是独立的。UDP 解决的是“数据报如何到达目标进程”，而一个面向用户的网络服务还需要回答另一个问题：服务器如何知道**当前请求属于哪个已经登录的用户**。浏览器的登录页、身份认证、Cookie 和服务器端 Session 正是围绕这个问题展开的。

3.1 一次登录请求如何变成持续会话

3.1.1 登录页与身份认证

课堂图中的典型流程如下：

1. 客户端请求登录页面 `login.html`；
2. 用户在页面中填写账号和密码；
3. 浏览器把登录请求发送给服务器；
4. 服务器取得账号和密码，并与用户数据库或其他认证系统比对；
5. 认证通过后，服务器为用户建立登录会话。

这里要区分两个概念：

- **认证（Authentication）**：确认“当前请求者是谁”；
- **会话（Session）**：认证成功后，在一段时间内记住“该用户已经登录”。

如果每个请求都重新提交账号和密码，不仅效率低，也会扩大密码泄漏的风险。因此，登录成功后需要一种机制让后续请求携带一个临时身份凭证。

```
ZL@VM-0-14-ubuntu:~$ ps ajx | grep sshd
      1     933     933     933 ?             -1 Ss       0   0:56 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 star
ups
    933  186429  186429  186429 ?             -1 Ss       0   0:00 sshd: ZL [priv]
 186429  186511  186429  186429 ?             -1 S     1002   0:00 sshd: ZL@pts/15
    933  204853  204853  204853 ?             -1 Ss       0   0:00 sshd: ZL [priv]
    933  204854  204854  204854 ?             -1 Ss       0   0:00 sshd: ZL [priv]
 204854  205074  204854  204854 ?             -1 S     1002   0:00 sshd: ZL@pts/6
 204853  205164  204853  204853 ?             -1 S     1002   0:00 sshd: ZL@pts/16
 205075  205434  205433  205075 pts/6     205433 S+    1002   0:00 grep --color=auto sshd
```

```
ZL@VM-0-14-ubuntu:~$ cat /etc/password
cat: /etc/password: No such file or directory
ZL@VM-0-14-ubuntu:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
```

3.1.2 服务器端如何保存登录状态

认证成功后，服务器通常会构造一份结构化会话信息，例如：

> 账号
> 用户角色
> 登录时间
> 最后访问时间
> 过期时间
> 其他与业务有关的登录状态

这些信息可以保存在服务器的文件中，也可以保存到 Redis 等键值存储中。随后服务器生成一个难以猜测且唯一的 `session_id`，把它作为查找这份会话信息的键。
可以概括为：

```
session_id -> 服务器端会话数据
```

浏览器并不需要知道服务器端保存了哪些完整状态，它只需要保存能够找回这份状态的`session_id`。

3.1.3 Set-Cookie 与后续请求

服务器认证成功后，会在 HTTP 响应头中通知浏览器保存会话标识：

```
Set-Cookie: session_id=XXXX
```

浏览器之后的请求会把它自动带回：

```
Cookie: session_id=XXXX
```

服务器读取 Cookie 中的`session_id`，再到文件或 Redis 中查找对应的会话数据，从而判断当前用户是谁、是否仍处于登录状态。

因此：

> session + cookie = 会话管理

可以进一步拆成：

- **Cookie 在客户端**负责保存会话标识；
- **Session 数据在服务器端**负责保存真实登录状态；
- **session_id**把客户端 Cookie 与服务器端状态连接起来。

3.2 Cookie 与 Session 的职责边界

3.2.1 不建议把私密信息直接放进 Cookie

如果采用“纯 Cookie”方式，把账号、密码或大量用户隐私数据直接保存在客户端 Cookie 中，一旦这些内容被读取或篡改，就容易产生用户数据泄漏和身份冒充问题。

更合理的做法是只在 Cookie 中保存临时的 `session_id`，把权限、账号资料等敏感状态留在服务器端。这样客户端拿到的只是一个引用，而不是完整的用户数据。

即使如此，`session_id` 本身也相当于临时钥匙，仍然需要保护：

- 登录和后续请求应使用 HTTPS，避免账号密码和 Cookie 被网络窃听；
- Cookie 应设置合适的过期时间；
- 敏感 Cookie 应使用 `HttpOnly`，避免被普通前端脚本读取；
- 使用 HTTPS 的场景应设置 `Secure`，限制 Cookie 只通过安全连接发送；
- 根据业务需要设置 `SameSite`，降低跨站请求冒用 Cookie 的风险；
- 登录成功或权限变化后可以重新生成 `session_id`，减少会话固定攻击风险。

这里说明的是防御原则，不表示设置几个 Cookie 属性就能解决所有安全问题。认证、授权、输入校验、服务端日志和会话失效策略仍然需要共同配合。

3.2.2 注销、过期与失效

用户注销时，服务器应删除对应的会话数据，浏览器也应清除 Cookie。只删除客户端 Cookie 而保留服务器端 Session，旧 `session_id` 在过期前仍可能被再次使用，因此更稳妥的做法是：

1. 服务端把 Session 标记为失效或直接删除；
2. 客户端清除对应 Cookie；
3. 后续携带旧 `session_id` 的请求统一拒绝。

会话还会因为超过空闲时间、超过绝对有效期、密码修改或管理员强制下线而失效。强调“主动权在服务端”，原因就在这里：浏览器可以保存和提交凭证，但用户是否仍然登录、拥有哪些权限，最终必须由服务器判断。

3.3 Web 会话与操作系统会话的联系

Web 中的 Session 和 Linux 中的 Session 不是同一个数据结构，但二者都体现了“先识别、再关联”的思想：

- Web 会话用 `session_id` 把多次 HTTP 请求关联为同一个用户；
- Linux 用 `SID` 把一次登录所产生的终端、Shell 和作业关联为同一个会话。

理解了这一层对应关系，就能进一步理解：一个网络服务如果依附在某个用户登录会话下，就会受到用户注销、终端关闭等事件的影响；要成为长期独立运行的服务，还需要在操作系统层面进行守护进程化。

4. 终端会话与进程管理

4.1 进程组、作业与 Session

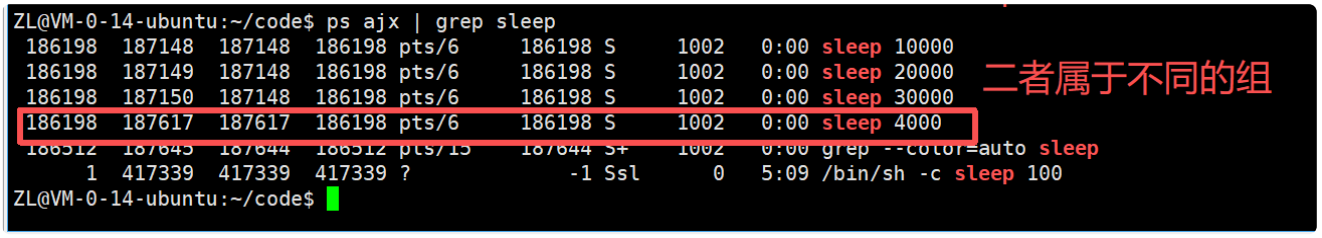
4.1.1 进程组与作业

进程组是一组进程的集合，每个进程组都有一个进程组 ID（PGID）。Shell 为了方便用户管理命令，会把一组相关进程组织成一个“作业”。

可以把它们比作“一枚硬币的两面”：

- **作业**更强调用户希望完成什么任务；
- **进程组**更强调这个任务由哪些进程共同完成、如何接收信号。

例如执行 `./netcal` 或启动一个 HTTP 服务时，Shell 会把相关进程放入一个进程组。前台进程组可以接收终端输入，也可以接收 `Ctrl+C`、`Ctrl+Z` 等终端信号。

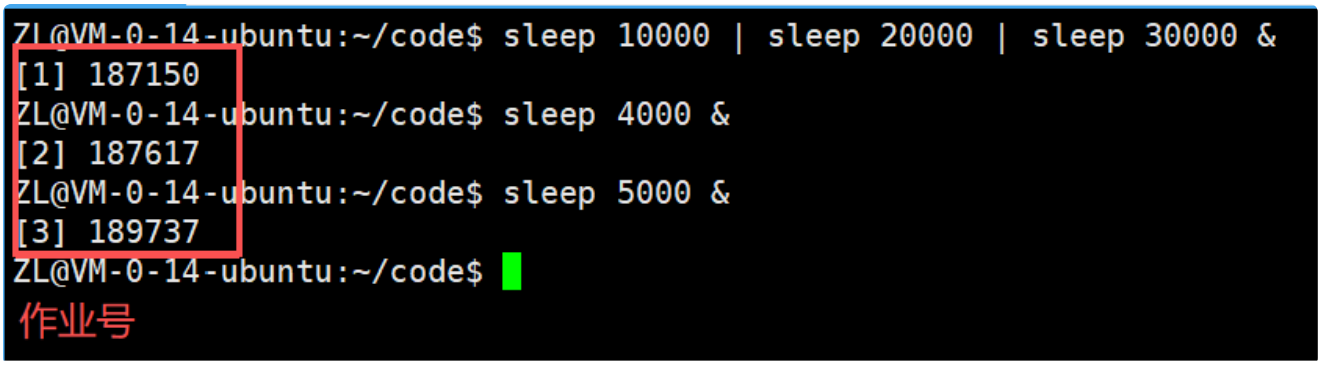


4.1.2 作业号与 jobs

Shell 会给后台作业分配作业号，用 `jobs` 可以查看它们：

```
jobs
```

例如 `[1]+ Running ./server` 中的 `1` 是作业号，`+` 是当前作业标记。作业号是 Shell 为了方便管理而使用的短编号，进程的 PID 和 PGID 则是操作系统层面的编号，不能混为一谈。




```
ZL@VM-0-14-ubuntu:~/code$ sleep 10000 | sleep 20000 | sleep 30000 &
[1] 187150
ZL@VM-0-14-ubuntu:~/code$ sleep 4000 &
[2] 187617
ZL@VM-0-14-ubuntu:~/code$ sleep 5000 &
[3] 189737
ZL@VM-0-14-ubuntu:~/code$ jobs
[1]   Running                 sleep 10000 | sleep 20000 | sleep 30000 &
[2]-  Running                 sleep 4000 &
[3]+  Running                 sleep 5000 &
ZL@VM-0-14-ubuntu:~/code$
```

4.1.3 进程组与进程组长

进程组不会因为组长进程单独退出就立刻消失。只要组内仍有进程存在，这个进程组就可以继续存在。换一种说法：

```
进程组 != 进程组长
```

组长只是让进程组使用了自己的 PID 作为 PGID，并不意味着组长离开后全组立即被销毁。

4.2 前台与后台作业

4.2.1 一个终端会话的输入权

在同一个登录会话中，通常只允许一个作业位于前台，其余作业在后台运行。前台作业拥有从终端文件读取输入的权利，后台作业则适合运行时间较长、不需要用户持续输入的任务。

因此可以形成下面的选择原则：

- 短时间、需要观察输出或输入的任务放在前台；
- 长时间运行、不需要键盘交互的任务可以放到后台。

让任务在后台启动的常见方式是：

```
./httpserver &
```

4.2.2 fg、bg、Ctrl+C 与 Ctrl+Z

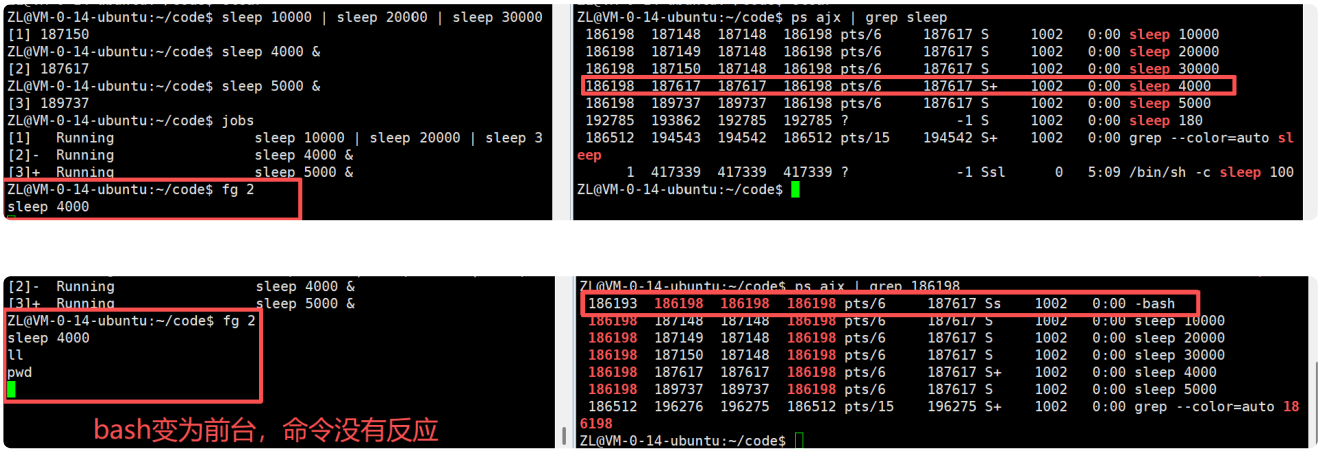
Shell 提供了几个常用控制命令：

```
fg %1       # 把 1 号作业放到前台
bg %1       # 让暂停的 1 号作业在后台继续运行
```

终端快捷键对应不同信号：

- `Ctrl+C`：通常向当前前台进程组发送 `SIGINT`，用于中断任务；
- `Ctrl+Z`：通常发送 `SIGTSTP`，暂停当前前台作业并把它交给 Shell 管理。

`fg` 把后台作业拉到前台后，Shell 本身转入后台；前台作业被暂停或结束后，Shell 再获得终端控制权，重新显示命令提示符。



4.3 SID 与终端登录会话

4.3.1 登录会创建 Session

每次用户登录，系统都会为其创建终端文件、Shell 进程和相关作业。这些内容共同构成一次 Session：

```
一次登录 -> 一个 Session -> 至少包含一个进程组
```

每个 Session 都有 SID（Session ID），通常由会话首进程的 PID 决定。登录 Shell 往往就是这个会话中的首个重要进程，因此会看到类似 `PID == SID == bash` 的现象。

Linux 下终端文件通常位于 `/dev/pts/`。终端同样遵循“一切皆文件”的思想：Shell 和前台进程通过终端文件读取键盘输入、写出显示内容。

```
###############
   PPID     PID    PGID     SID TTY        TPGID STAT   UID   TIME COMMAND
 376142  376145  376145  376145 pts/6     379400 Ss    1002   0:00 -bash
 377628  377629  377629  377629 pts/15    377629 Ss+   1002   0:00 -bash
      1 1835700 1835700 1835700 pts/11   1835700 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
      1 2714641 2714641 2714641 pts/9    2714641 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
      1 2720079 2720079 2720079 pts/10   2720079 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
total 0
drwxr-xr-x  2 root root       0 Jul  3 23:15 ./
drwxr-xr-x 19 root root    3960 Jul  3 23:15 ../
crw--w----  1 root tty  136,  1 Jul  4 10:11 1
crw--w----  1 ZL   tty  136, 10 Aug  7 10:16 10
crw--w----  1 ZL   tty  136, 11 Aug  3 21:46 11
crw--w----  1 ZL   tty  136, 15 Sep 18 08:56 15
crw--w----  1 root tty  136,  4 Jul  4 10:16 4
crw--w----  1 ZL   tty  136,  6 Sep 18 08:56 6
crw--w----  1 ZL   tty  136,  8 Aug  7 13:25 8
crw--w----  1 ZL   tty  136,  9 Aug  7 10:16 9
c---------  1 root root   5,  2 Jul  3 23:15 ptmx
```

可以通过下面的命令观察进程、终端文件和 SID 的变化：

```
while :; do
    ps ajx | head -1
    ps ajx | grep bash | grep -v grep
    ll /dev/pts
    sleep 1
    echo "###############"
done
```

4.3.2 注销会话为什么会影响服务

当用户注销或终端关闭时，该 Session 中原本依附于终端的作业可能收到挂断或终止相关信号，具体行为取决于进程是否忽略信号、是否处于不同进程组以及由谁启动。

如果 HTTP 服务只是某个登录 Shell 中的普通前台或后台作业，它就仍然与这个会话存在关联。用户退出后，服务可能被终止；即使没有立即退出，也可能因为终端、工作目录或标准输入输出失效而无法稳定工作。

这不是一个合格的网络服务生命周期。真正长期运行的网络服务应脱离某个用户的登录会话。

5. 网络服务守护进程化

5.1 为什么要创建独立 Session

UDP 服务器、HTTP 服务器等网络进程通常在后台长期运行，不应该因为某个用户退出终端而停止。因此，需要让服务：

- 脱离原来的控制终端；
- 成为新 Session 的会话首进程；
- 不再依赖登录 Shell 的生命周期；
- 明确管理标准输入、标准输出和错误日志；
- 使用稳定的工作目录和资源路径。

这个把普通进程转换为后台服务进程的过程，就是守护进程化（daemonization）。

5.2 setsid 与守护进程

5.2.1 setsid 做了什么

`setsid()` 会创建一个新 Session，并使调用它的进程成为新会话的会话首进程，同时成为新进程组的组长。

调用前有一个限制：如果当前进程已经是某个进程组的组长，就不能直接调用 `setsid()`。常见的处理方式是让父进程 `fork()` 出子进程，父进程立即退出，再由子进程调用 `setsid()`：

```
父进程
  └── fork()
        ├── 父进程退出
        └── 子进程 setsid()
              └── 成为新会话首进程
```

成功脱离并运行在独立会话中的服务进程，就是通常所说的守护进程（daemon）。课堂手记也把它称为“孤儿进程”或“精灵进程”：父进程退出后，它不再依附原来的父子关系和登录会话。

`setsid()` 创建的是新的 Session，不会自动创建新的 Shell，也不会创建新的终端文件。登录时创建终端文件和 Shell，与服务守护进程化是两个不同的过程。

```
setsid(2)                                                  System Calls Manual

NAME
       setsid - creates a session and sets the process group ID

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       pid_t setsid(void);
```

5.2.2 守护进程的工程含义

守护进程不仅是一个“转到后台”的动作，还意味着服务生命周期的独立：

- 不受单个终端关闭影响；
- 不受普通用户注销影响；
- 可以持续接收网络请求并维护服务器状态；
- 可以按照系统服务方式启动、停止和管理；
- 出错后应通过日志和监控发现，而不是依赖终端输出。

5.3 标准流与日志

5.3.1 重定向标准输入输出

守护进程不能继续依赖原来的终端标准流。通常会把：

- 标准输入重定向到 `/dev/null`；
- 标准输出重定向到日志文件或日志系统；
- 标准错误重定向到错误日志或日志系统。

`/dev/null` 会丢弃写入内容，读取时不会得到真实业务数据。另一个特殊文件 `/dev/zero` 则会持续提供零字节，适合需要大量零字节数据的场景；它提供的不是随机数据，这一点需要注意。

直接把所有标准流关闭虽然看起来“干净”，但后续库函数或错误处理再次写入这些描述符时可能不断失败。更可靠的方式是把它们重定向到明确位置。

```
ZL@VM-0-14-ubuntu:~$ ll /dev/null
crw-rw-rw- 1 root root 1, 3 Jul  3 23:15 /dev/null
ZL@VM-0-14-ubuntu:~$ ll /dev/zero
crw-rw-rw- 1 root root 1, 5 Jul  3 23:15 /dev/zero
ZL@VM-0-14-ubuntu:~$
```

5.3.2 通过日志观察服务

守护进程没有前台终端可看，因此日志是最重要的诊断入口。日志至少应记录：

- 服务启动时间和监听端口；
- socket 创建、绑定是否成功；
- 接收和处理请求时的异常；
- 配置加载失败和关键资源缺失；
- 服务停止原因。

强调“真正的调试方式是通过查看日志并打印”，原因就是守护进程必须能在无人值守的情况下留下可追踪的运行证据。

5.4 部署目录与路径管理

5.4.1 模块化目录

一个可部署的服务不应把程序、配置、日志和静态资源混在一起，所以给出的结构应该是：

```
bin/httpserver
conf/http.conf
log/
wwwroot/
```

各部分的职责是：

- `bin/` 保存可执行程序；
- `conf/` 保存配置文件；

- `log/` 保存运行日志；
- `wwwroot/` 保存对外提供的静态资源。

部署时可以打包并解压到目标目录，再根据实际位置调整配置文件。这样服务程序与运行时资源保持清晰边界。

5.4.2 使用稳定的起始路径

守护进程启动后的工作目录可能不再是用户登录时的目录，因此不能依赖相对路径碰巧有效。服务必须有一个固定的资源定位起点，并优先通过绝对路径或配置项找到：

- 配置文件；
- 日志目录；
- 网页根目录；
- 数据文件和证书文件。

否则同一个程序从不同目录启动时，可能表现出完全不同甚至看似“无法解释”的行为。

```
ZL@VM-0-14-ubuntu:~/code/lesson41/HTTP$ ps ajx | grep httpserver && ps ajx | head -1
      1  427224  427224  427224 ?             -1 Ss    1002   0:00 httpserver 8080
 421690  428424  428423  421690 pts/6     428423 S+    1002   0:00 grep --color=auto httpserver
   PPID     PID    PGID     SID TTY        TPGID STAT   UID   TIME COMMAND
```

```
lrwxrwxrwx  1 ZL ZL  0 Sep 18 12:10 cwd -> /home/ZL/code/lesson41/HTTP
-r--------  1 ZL ZL  0 Sep 18 12:10 environ
lrwxrwxrwx  1 ZL ZL  0 Sep 18 12:10 exe -> /usr/bin/httpserver
```

```
ZL@VM-0-14-ubuntu:~/code/lesson41/HTTP$ ls /proc/427224/fd -l
total 0
lrwx------ 1 ZL ZL 64 Sep 18 12:18 0 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 1 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 2 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 3 -> 'socket:[139005925]'
```

```
daemon(3)

NAME
       daemon - run in the background

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int daemon(int nochdir, int noclose);
```

5.5 从登录 Session 到独立网络服务

完整的演进关系可以概括为：

1. **Web 登录**通过认证建立用户身份，使用 Cookie 中的 `session_id` 关联服务器端 Session；
2. **终端登录**创建操作系统 Session，Shell、终端文件和作业都归属于该 Session；
3. **普通启动的服务器**仍可能受用户注销和终端关闭影响；
4. **调用 `setsid()` 后**，服务进入独立 Session，不再依附原登录终端；
5. **重定向标准流并配置固定资源路径后**，服务才能稳定记录日志、读取配置并长期运行；
6. **UDP 服务本身**只负责数据报收发，而守护进程化解决的是服务进程的生命周期和部署问题。

因此，协议层与操作系统层承担不同职责：UDP 决定数据报如何到达端口对应的进程，Session 与 Cookie 决定多次请求如何关联到同一用户，进程、进程组、Session 和守护进程则决定服务进程能否稳定、独立地长期运行。